```tsx
import React from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface SkeletonProps extends React.HTMLAttributes<HTMLDivElement> {
  /**
   * Visual variant of the skeleton
   * @default "text"
   */
  variant?: "text" | "circular" | "rectangular" | "rounded";
  /**
   * Width of the skeleton
   * @default "100%"
   */
  width?: string | number;
  /**
   * Height of the skeleton
   * @default "1rem"
   */
  height?: string | number;
  /**
   * Whether the skeleton should animate
   * @default "pulse"
   */
  animation?: "pulse" | "wave" | "none";
  /**
   * Border radius for rectangular variant
   * @default "0.375rem"
   */
  radius?: string | number;
  /**
   * Base color for the skeleton
   * @default "muted"
   */
  baseColor?: "muted" | "muted-foreground/10" | "border" | "primary/10";
}

/**
 * Skeleton - A versatile skeleton loading placeholder component
 *
 * Provides multiple visual variants for different content types:
 * - text: Simulates text lines with varying widths
 * - circular: Circular placeholder for avatars, badges
 * - rectangular: Rectangular placeholder for cards, images
 * - rounded: Rounded rectangular placeholder for buttons, chips
 *
```

```
 * Supports multiple animation types:
 * - pulse: Subtle opacity pulse animation (default)
 * - wave: Shimmer wave animation
 * - none: Static placeholder without animation
 *
 * @example
 * ```tsx
 * // Text skeleton
 * <Skeleton variant="text" width="80%" />
 *
 * // Circular avatar skeleton
 * <Skeleton variant="circular" width={40} height={40} />
 *
 * // Card skeleton with wave animation
 * <Skeleton variant="rectangular" width="100%" height={200} animation="wave" />
 *
 * // Button skeleton
 * <Skeleton variant="rounded" width={120} height={40} radius="9999px" />
 * ```
 */
export const Skeleton = React.forwardRef<HTMLDivElement, SkeletonProps>(
  (
    {
      className,
      variant = "text",
      width = "100%",
      height = "1rem",
      animation = "pulse",
      radius = "0.375rem",
      baseColor = "muted",
      style,
      ...props
    },
    ref,
  ) => {
    const widthValue = typeof width === "number" ? `${width}px` : width;
    const heightValue = typeof height === "number" ? `${height}px` : height;
    const radiusValue = typeof radius === "number" ? `${radius}px` : radius;

    const variantClasses = {
      circular: "rounded-full",
      rounded: `rounded-[${radiusValue}]`,
      rectangular: `rounded-[${radiusValue}]`,
      text: "rounded-[0.25rem]",
    }[variant];

    const animationClasses = {
      pulse: "animate-pulse duration-1000",
      wave: "animate-[skeleton-wave_1.5s_ease-in-out_infinite] bg-gradient-to-r
from-[hsl(var(--muted))] via-[hsl(var(--muted-foreground/10))]
```

```
to-[hsl(var(--muted))] bg-[length:200%_100%]",
      none: "",
    }[animation];

    const baseColorClass = `bg-[hsl(var(--${baseColor}))]`;

    return (
      <div
        ref={ref}
        className={cn(
          "skeleton",
          variantClasses,
          animationClasses,
          baseColorClass,
          className,
        )}
        style={{
          width: widthValue,
          height: heightValue,
          ...style,
        }}
        {...props}
      />
    );
  },
);

Skeleton.displayName = "Skeleton";

/**
 * SkeletonText - Pre-configured skeleton for text content
 * Renders multiple lines with varying widths to simulate paragraph text
 */
export interface SkeletonTextProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Number of text lines to render
   * @default 3
   */
  lines?: number;
  /**
   * Maximum width of the text block
   * @default "100%"
   */
  maxWidth?: string | number;
  /**
   * Spacing between lines
   * @default "0.5rem"
```

```
   */
  lineGap?: string | number;
  /**
   * Whether the last line should be shorter (more realistic)
   * @default true
   */
  lastLineShort?: boolean;
  /**
   * Approximate width of the last line when lastLineShort is true
   * @default "60%"
   */
  lastLineWidth?: string | number;
}

export const SkeletonText = React.forwardRef<HTMLDivElement, SkeletonTextProps>(
  (
    {
      className,
      lines = 3,
      maxWidth = "100%",
      lineGap = "0.5rem",
      lastLineShort = true,
      lastLineWidth = "60%",
      animation = "pulse",
      baseColor = "muted",
      radius = "0.25rem",
      style,
      ...props
    },
    ref,
  ) => {
    const maxWidthValue =
      typeof maxWidth === "number" ? `${maxWidth}px` : maxWidth;
    const lineGapValue = typeof lineGap === "number" ? `${lineGap}px` : lineGap;
    const lastLineWidthValue =
      typeof lastLineWidth === "number" ? `${lastLineWidth}px` : lastLineWidth;

    const lineHeight = "1rem";

    return (
      <div
        ref={ref}
        className={cn("skeleton-text-container", className)}
        style={{
          maxWidth: maxWidthValue,
          display: "flex",
          flexDirection: "column",
          gap: lineGapValue,
          ...style,
        }}
```

```
        {...props}
      >
        {Array.from({ length: lines }).map((_, index) => {
          const isLastLine = index === lines - 1;
          const width =
            isLastLine && lastLineShort ? lastLineWidthValue : "100%";

          return (
            <Skeleton
              key={index}
              variant="text"
              width={width}
              height={lineHeight}
              animation={animation}
              baseColor={baseColor}
              radius={radius}
            />
          );
        })}
      </div>
    );
  },
);

SkeletonText.displayName = "SkeletonText";

/**
 * SkeletonCard - Pre-configured skeleton for card layouts
 * Renders a card skeleton with image placeholder, title, and text lines
 */
export interface SkeletonCardProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Whether to show image placeholder
   * @default true
   */
  withImage?: boolean;
  /**
   * Height of the image placeholder
   * @default "200px"
   */
  imageHeight?: string | number;
  /**
   * Number of text lines in the card body
   * @default 3
   */
  lines?: number;
  /**
```

```
   * Whether to show action buttons placeholder
   * @default false
   */
  withActions?: boolean;
  /**
   * Card border radius
   * @default "0.5rem"
   */
  radius?: string | number;
}

export const SkeletonCard = React.forwardRef<HTMLDivElement, SkeletonCardProps>(
  (
    {
      className,
      withImage = true,
      imageHeight = "200px",
      lines = 3,
      withActions = false,
      radius = "0.5rem",
      animation = "pulse",
      baseColor = "muted",
      style,
      ...props
    },
    ref,
  ) => {
    const radiusValue = typeof radius === "number" ? `${radius}px` : radius;
    const imageHeightValue =
      typeof imageHeight === "number" ? `${imageHeight}px` : imageHeight;

    return (
      <div
        ref={ref}
        className={cn("skeleton-card", className)}
        style={{
          borderRadius: radiusValue,
          overflow: "hidden",
          backgroundColor: `hsl(var(--${baseColor}))`,
          ...style,
        }}
        {...props}
      >
        {withImage && (
          <Skeleton
            variant="rectangular"
            width="100%"
            height={imageHeightValue}
            animation={animation}
            baseColor={baseColor}
```

```
            radius="0"
          />
        )}
        <div className="skeleton-card-content" style={{ padding: "1rem" }}>
          <Skeleton
            variant="text"
            width="60%"
            height="1.25rem"
            animation={animation}
            baseColor={baseColor}
            radius={radius}
          />
          <SkeletonText
            lines={lines}
            animation={animation}
            baseColor={baseColor}
            radius={radius}
            lineGap="0.375rem"
          />
          {withActions && (
            <div
              className="skeleton-card-actions"
              style={{ marginTop: "1rem", display: "flex", gap: "0.5rem" }}
            >
              <Skeleton
                variant="rounded"
                width={100}
                height={40}
                animation={animation}
                baseColor={baseColor}
                radius="9999px"
              />
              <Skeleton
                variant="rounded"
                width={100}
                height={40}
                animation={animation}
                baseColor={baseColor}
                radius="9999px"
              />
            </div>
          )}
        </div>
      </div>
    );
  },
);

SkeletonCard.displayName = "SkeletonCard";
```

```
/**
 * SkeletonAvatar - Pre-configured skeleton for avatar placeholders
 */
export interface SkeletonAvatarProps extends Omit<
  SkeletonProps,
  "variant" | "width" | "height"
> {
  /**
   * Size of the avatar
   * @default "md"
   */
  size?: "xs" | "sm" | "md" | "lg" | "xl" | "2xl";
  /**
   * Custom size in pixels (overrides size prop)
   */
  sizePx?: number;
}

const avatarSizes = {
  xs: 24,
  sm: 32,
  md: 40,
  lg: 48,
  xl: 64,
  "2xl": 96,
} as const;

export const SkeletonAvatar = React.forwardRef<
  HTMLDivElement,
  SkeletonAvatarProps
>(
  (
    {
      className,
      size = "md",
      sizePx,
      animation = "pulse",
      baseColor = "muted",
      style,
      ...props
    },
    ref,
  ) => {
    const sizeValue = sizePx ?? avatarSizes[size];

    return (
      <Skeleton
        ref={ref}
        className={cn("skeleton-avatar", className)}
        variant="circular"
```

```
        width={sizeValue}
        height={sizeValue}
        animation={animation}
        baseColor={baseColor}
        style={style}
        {...props}
      />
    );
  },
);

SkeletonAvatar.displayName = "SkeletonAvatar";

/**
 * SkeletonButton - Pre-configured skeleton for button placeholders
 */
export interface SkeletonButtonProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Button size variant
   * @default "md"
   */
  size?: "sm" | "md" | "lg" | "icon";
  /**
   * Custom width (overrides size)
   */
  width?: string | number;
  /**
   * Custom height (overrides size)
   */
  height?: string | number;
  /**
   * Border radius
   * @default "0.375rem"
   */
  radius?: string | number;
}

const buttonSizes = {
  sm: { height: 32, width: "auto", minWidth: 64 },
  md: { height: 40, width: "auto", minWidth: 80 },
  lg: { height: 48, width: "auto", minWidth: 96 },
  icon: { height: 40, width: 40, minWidth: 40 },
} as const;

export const SkeletonButton = React.forwardRef<
  HTMLDivElement,
  SkeletonButtonProps
```

```
>(
  (
    {
      className,
      size = "md",
      width,
      height,
      animation = "pulse",
      baseColor = "muted",
      radius = "0.375rem",
      style,
      ...props
    },
    ref,
  ) => {
    const sizeConfig = buttonSizes[size];
    const widthValue = width ?? sizeConfig.width;
    const heightValue = height ?? sizeConfig.height;
    const minWidthValue = sizeConfig.minWidth;

    return (
      <Skeleton
        ref={ref}
        className={cn("skeleton-button", className)}
        variant="rounded"
        width={widthValue}
        height={heightValue}
        animation={animation}
        baseColor={baseColor}
        radius={radius}
        style={{
          minWidth:
            typeof minWidthValue === "number"
              ? `${minWidthValue}px`
              : minWidthValue,
          ...style,
        }}
        {...props}
      />
    );
  },
);

SkeletonButton.displayName = "SkeletonButton";

/**
 * SkeletonInput - Pre-configured skeleton for input field placeholders
 */
export interface SkeletonInputProps extends Omit<
  SkeletonProps,
```

```
  "variant" | "height"
> {
  /**
   * Input size variant
   * @default "md"
   */
  size?: "sm" | "md" | "lg";
  /**
   * Custom width
   * @default "100%"
   */
  width?: string | number;
}

const inputSizes = {
  sm: { height: 32 },
  md: { height: 40 },
  lg: { height: 48 },
} as const;

export const SkeletonInput = React.forwardRef<
  HTMLDivElement,
  SkeletonInputProps
>(
  (
    {
      className,
      size = "md",
      width = "100%",
      animation = "pulse",
      baseColor = "muted",
      radius = "0.375rem",
      style,
      ...props
    },
    ref,
  ) => {
    const heightValue = inputSizes[size].height;

    return (
      <Skeleton
        ref={ref}
        className={cn("skeleton-input", className)}
        variant="rounded"
        width={width}
        height={heightValue}
        animation={animation}
        baseColor={baseColor}
        radius={radius}
        style={style}
```

```
        {...props}
      />
    );
  },
);

SkeletonInput.displayName = "SkeletonInput";

/**
 * SkeletonList - Pre-configured skeleton for list item placeholders
 */
export interface SkeletonListProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Number of list items to render
   * @default 5
   */
  items?: number;
  /**
   * Whether each item has an avatar/icon placeholder
   * @default true
   */
  withAvatar?: boolean;
  /**
   * Number of text lines per item
   * @default 2
   */
  linesPerItem?: number;
  /**
   * Gap between list items
   * @default "0.75rem"
   */
  itemGap?: string | number;
}

export const SkeletonList = React.forwardRef<HTMLDivElement, SkeletonListProps>(
  (
    {
      className,
      items = 5,
      withAvatar = true,
      linesPerItem = 2,
      itemGap = "0.75rem",
      animation = "pulse",
      baseColor = "muted",
      radius = "0.375rem",
      style,
      ...props
```

```
  },
  ref,
) => {
  const itemGapValue = typeof itemGap === "number" ? `${itemGap}px` : itemGap;

  return (
    <div
      ref={ref}
      className={cn("skeleton-list", className)}
      style={{
        display: "flex",
        flexDirection: "column",
        gap: itemGapValue,
        ...style,
      }}
      {...props}
    >
      {Array.from({ length: items }).map((_, index) => (
        <div
          key={index}
          className="skeleton-list-item"
          style={{
            display: "flex",
            alignItems: "flex-start",
            gap: "0.75rem",
          }}
        >
          {withAvatar && (
            <SkeletonAvatar
              size="md"
              animation={animation}
              baseColor={baseColor}
            />
          )}
          <div style={{ flex: 1, minWidth: 0 }}>
            <SkeletonText
              lines={linesPerItem}
              animation={animation}
              baseColor={baseColor}
              radius={radius}
              lineGap="0.25rem"
              lastLineShort
              lastLineWidth="70%"
            />
          </div>
        </div>
      ))}
    </div>
  );
},
```

```
);

SkeletonList.displayName = "SkeletonList";

/**
 * SkeletonTable - Pre-configured skeleton for table row placeholders
 */
export interface SkeletonTableProps extends Omit<
  SkeletonProps,
  "variant" | "height" | "width"
> {
  /**
   * Number of rows to render
   * @default 5
   */
  rows?: number;
  /**
   * Number of columns
   * @default 4
   */
  columns?: number;
  /**
   * Whether to show header row skeleton
   * @default true
   */
  withHeader?: boolean;
  /**
   * Column widths (percentage or pixel values)
   */
  columnWidths?: (string | number)[];
}

export const SkeletonTable = React.forwardRef<
  HTMLDivElement,
  SkeletonTableProps
>(
  (
    {
      className,
      rows = 5,
      columns = 4,
      withHeader = true,
      columnWidths,
      animation = "pulse",
      baseColor = "muted",
      radius = "0.25rem",
      style,
      ...props
    },
    ref,
```

```
) => {
  const defaultColumnWidth = 100 / columns;
  const colWidths =
    columnWidths?.length === columns
      ? columnWidths
      : Array(columns).fill(defaultColumnWidth);

  const renderRow = (isHeader: boolean, rowIndex: number) => (
    <div
      key={rowIndex}
      className={cn(
        "skeleton-table-row",
        isHeader && "skeleton-table-header",
      )}
      style={{
        display: "grid",
        gridTemplateColumns: colWidths
          .map((w) => (typeof w === "number" ? `${w}%` : w))
          .join(" "),
        gap: "1rem",
        padding: "0.75rem 1rem",
        ...(isHeader && { fontWeight: 600 }),
      }}
    >
      {colWidths.map((_, colIndex) => (
        <Skeleton
          key={colIndex}
          variant="text"
          width="100%"
          height={isHeader ? "1rem" : "0.875rem"}
          animation={animation}
          baseColor={baseColor}
          radius={radius}
        />
      ))}
    </div>
  );

  return (
    <div
      ref={ref}
      className={cn("skeleton-table", className)}
      style={{
        borderRadius: radius,
        overflow: "hidden",
        backgroundColor: `hsl(var(--${baseColor}))`,
        ...style,
      }}
      {...props}
    >
```

```
        {withHeader && renderRow(true, -1)}
        {Array.from({ length: rows }).map((_, index) =>
          renderRow(false, index),
        )}
      </div>
    );
  },
);

SkeletonTable.displayName = "SkeletonTable";

export default Skeleton;
```